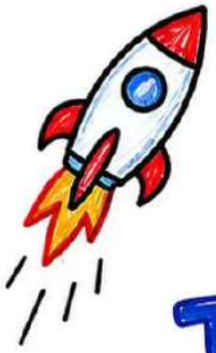
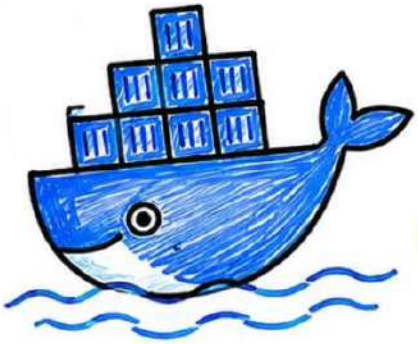
Slide 1 – Cover



TOP 10 DOCKER INTERVIEW QUESTIONS

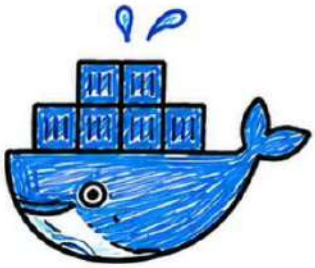


With Real-World Scenarios



Ace Your Next DevOps Interview 100





Slide 2 - Question 1

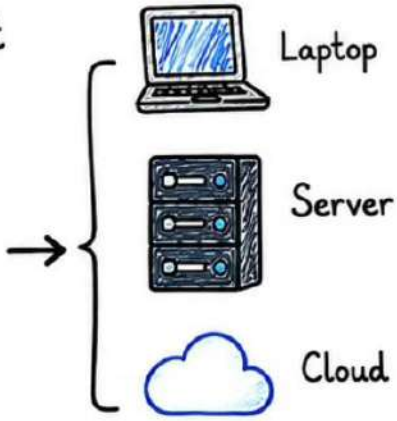


Q1. What is Docker?

Answer: Docker is a containerization platform that packages an application and all its dependencies into lightweight, portable containers, ensuring consistent execution across different environments.



Container



Runs Everywhere Consistently

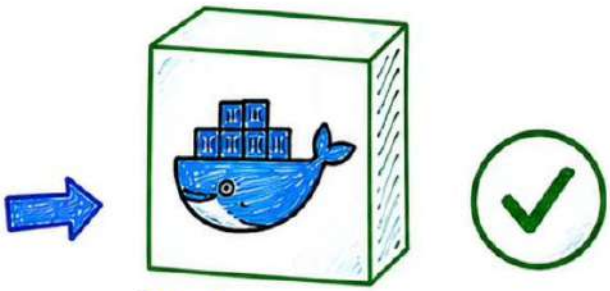
Real-World Scenario:



Developer's Laptop



Fails in production due to different Java versions



Docker packages everything together

Ensuring identical behavior everywhere. 100





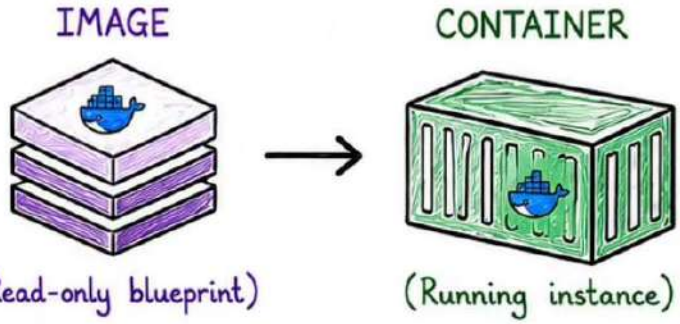
Slide 3 - Question 2



Q2. What is the difference between a Docker Image and a Docker Container?

Answer:

- Docker Image → Read-only blueprint
- Docker Container → Running instance of an image



DOCKER IMAGE



- Read-only
- Created using Dockerfile
- Stored in Docker Hub / Local
- Used to create containers
- Versionable and shareable

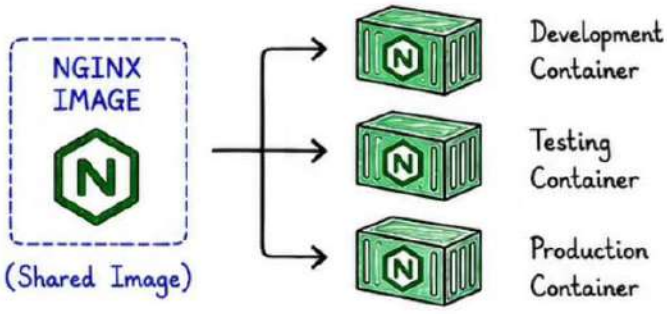
DOCKER CONTAINER

- Read-write
- Created from an image
- Running instance of an image
- Has its own writable layer
- Can be started, stopped, removed



Real-World Scenario:

One Nginx image can be used to run multiple independent web server containers for different environments.





One Image, Many Containers, Infinite Possibilities! 100



Slide 4 - Question 3

Q3. What happens when you run "docker run nginx"?



Answer:

Docker:

① Checks for the image locally

② Pulls it from Docker Hub if missing

③ Creates a container

④ Attaches networking

⑤ Starts the application



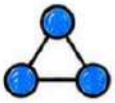
1. Checks for the image locally



2. Pulls it from Docker Hub if missing



3. Creates a container



4. Attaches networking



5. Starts the application

Real-World Scenario:



This process happens every time a new application container is deployed in CI/CD.

Key Takeaway



One simple command does a lot behind the scenes!

Docker = Simple outside, Powerful inside.





Remember: Build once, Run anywhere, Scale infinitely! 100



Slide 5 - Question 4



Q4. What is the difference between Docker Volumes and Bind Mounts?

Answer:

Named Volumes



- Docker-managed
- Stored in Docker's storage area
- Best for production
- Easier backup and portability
- Works on all environments

Example:

```
docker run -v mydata:/var/lib/mysql mysql
```

Benefits:

- ☑ Data managed by Docker
- ☑ Easy to migrate
- ☑ More secure and reliable

Bind Mounts



- Host-managed
- Directly maps a file or directory from host to container
- Best for development
- Changes on host reflect immediately in container

Example:

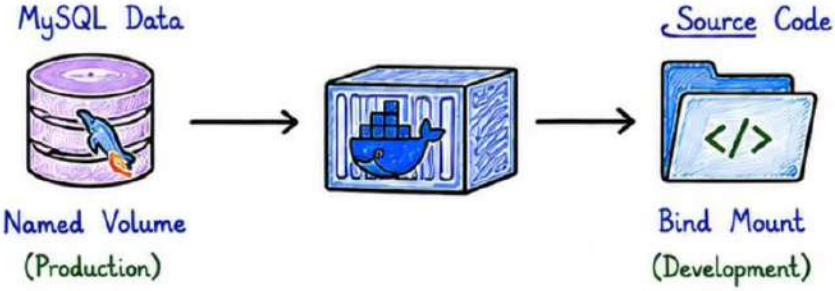
```
docker run -v /home/user/code:/app code-app
```

Benefits:

- ☑ Live code changes
- ☑ Easy debugging
- ☑ Full control over files

Real-World Scenario:

Use a Named Volume for MySQL data and a Bind Mount for live source code changes during development.



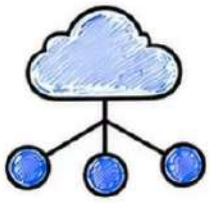


Choose the right mount based on your use case! 100





Slide 6 - Question 5



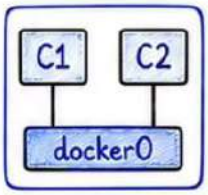
Q5. Explain Docker Networking.

Answer:

Docker provides different network drivers to enable communication between containers and with the outside world.

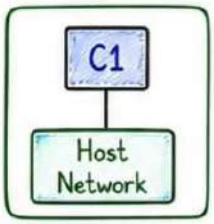
Types of Docker Networks

1. Bridge



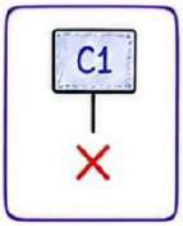
Default network driver. Containers communicate on the same Docker host.

2. Host



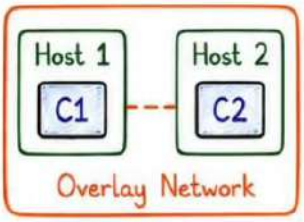
Container shares the host's network stack. No isolation.

3. None



Disables all networking for a container.

4. Overlay

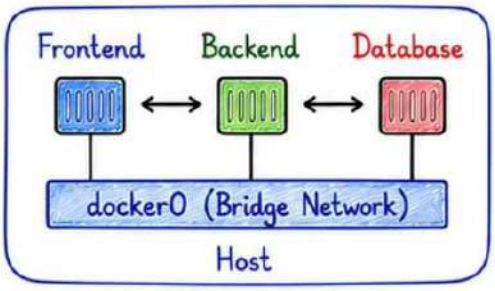


Connects containers across multiple Docker hosts.

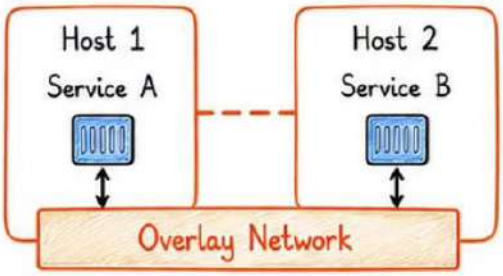
Real-World Scenario:

A frontend, backend, and database containers communicate over a Bridge network on the same host. For multi-host microservices communication, we use an Overlay network.

Single Host (Bridge Network)



Multiple Hosts (Overlay Network)





Key Takeaway

Choose the right network driver based on your application architecture and communication needs.

Quick Tip



Use `docker network ls` to list all networks.



Right Network = Better Communication + Security + Performance 100



Slide 7 - Question 6

Q6. What is a Dockerfile?



Dockerfile

Answer: A Dockerfile is a text file containing instructions to automatically build Docker images.

```
FROM openjdk:17-jre-slim          → Base Image
WORKDIR /app                      → Set Working Directory
COPY target/app.jar app.jar       → Copy Application
EXPOSE 8080                       → Expose Port
ENTRYPOINT ["java","-jar","app.jar"] → Run Application
```

Common Instructions:

- FROM → Base image
- WORKDIR → Set working directory
- COPY / ADD → Copy files into image
- RUN → Run commands
- EXPOSE → Expose container port
- ENV → Set environment variables
- CMD / ENTRYPOINT → Start the application

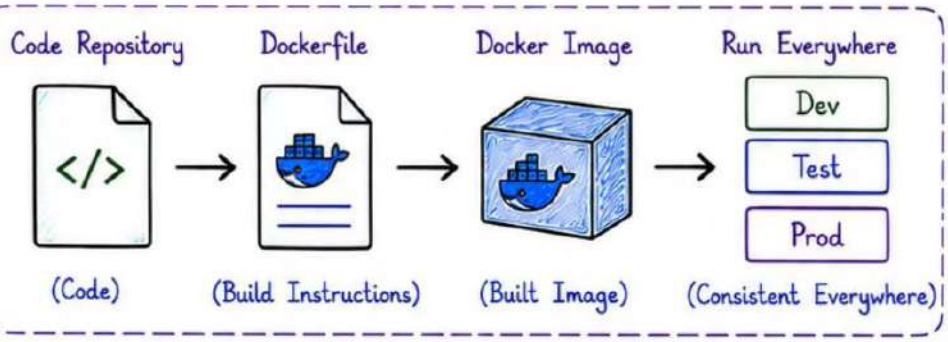
Benefits:

- ☑ Automates image creation
- ☑ Ensures consistency
- ☑ Version control friendly
- ☑ Easy to share and reuse
- ☑ Works in CI/CD pipelines



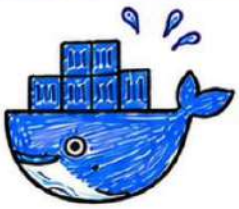
Real-World Scenario:

Every CI/CD pipeline uses Dockerfiles to build the same application image consistently across Dev, Test, and Prod environments.
This eliminates "it works on my machine" problems.





Dockerfile = Recipe for building your application image! 100



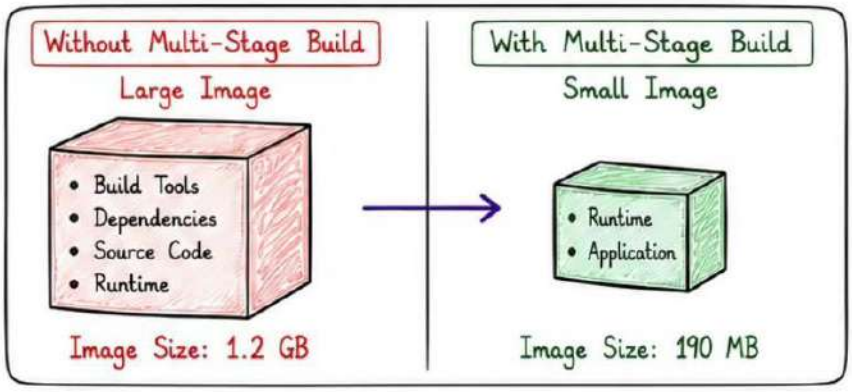
Slide 8 - Question 7



Q7. What are Multi-Stage Builds?

Answer:

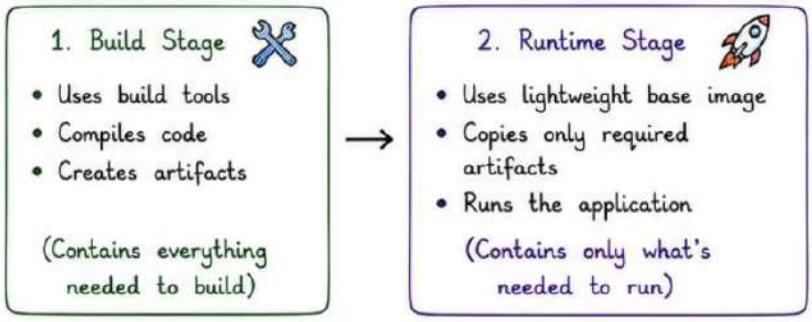
Multi-stage builds allow you to use multiple FROM statements in a Dockerfile to separate the build environment from the runtime environment.
This results in smaller, cleaner, and more secure images.



Example Dockerfile:

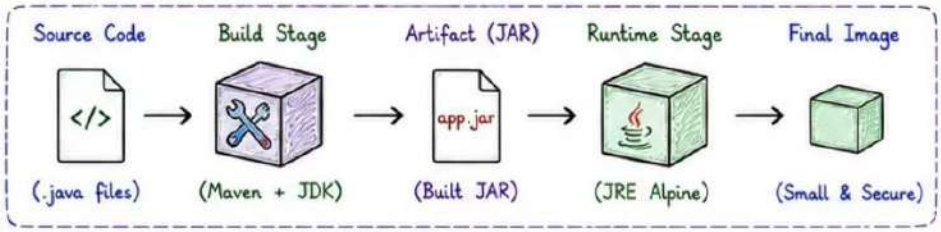
```
# ------------ Build Stage ----------
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
RUN  mvn clean package -DskipTests
# ----------- Runtime Stage -----------
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

How it works:



Real-World Scenario:

A Spring Boot application is built using Maven. Using multi-stage builds, we compile the app in the first stage and copy only the JAR file to a lightweight runtime image like alpine.
Result: Image size reduced from 1.2 GB to 190 MB!





Key Takeaways:

- ☑ Reduces image size
- ☑ Removes build tools from final image
- ☑ Speeds up downloads and deployments
- ☑ Improves security
- ☑ Best practice for production environments



Build smart, deploy fast, stay secure! Multi-Stage Builds are a Game Changer! 100



Slide 9 - Question 8



Q8. How do you troubleshoot a failing Docker container?

Answer: Use Docker commands to identify, analyze, and fix issues quickly.

Essential Docker Troubleshooting Commands

1. `docker ps -a`
   List all containers and their status.
2. `docker logs <container_id>`
   View logs to find errors.
3. `docker inspect <container_id>`
   Inspect container configuration and runtime details.
4. `docker exec -it <container_id> /bin/sh`
   Access container shell to debug inside.
5. `docker network inspect <network_name>`
   Check networking and connectivity issues.

Real-World Scenario

A Spring Boot application container keeps crashing (status: exited). Users can't access the app.

1. `docker ps -a` — Shows container exited (1)
2. `docker logs` — Reveals error: "Connection refused to DB"
3. `docker inspect` — Confirms wrong DB_HOST in env variable
4. `docker exec -it` — Verify env & test connection inside container
5. `docker network inspect` — Confirms network is correct
6. Fix configuration — Update DB_HOST, redeploy container, issue resolved!

Pro Tips

- ☑ Always start with logs.
- ☑ Inspect shows the actual runtime details.
- ☑ Most issues are config, network, or permission related.
- ☑ Reproduce the issue, then validate the fix.

Common Issues & What to Check

Issue	What to Check
Container Exits	Check logs, entrypoint, env vars
Cannot Connect	Check network, ports, firewall rules
DB Connection Failed	Check DB URL, credentials, network
Permission Denied	Check file permissions, user context

Key Takeaway: Logs tell you WHAT happened, Inspect tells you WHY it happened!

Troubleshoot smart. Fix faster. Keep your containers healthy! 100



Slide 10 - Question 9

Q9. What are Docker Best Practices for Production?



Answer:

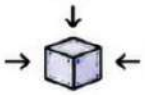
Following best practices helps you build secure, reliable, and efficient Docker applications in production.

Docker Best Practices

1 Use Official Base Images
Use trusted and minimal official images.



2 Keep Images Small
Use lightweight base images (alpine).
Remove unnecessary packages.



3 Don't Run as Root User
Create a non-root user and use it to run the application.



4 Scan Images for Vulnerabilities
Regularly scan images using tools like Trivy, Clair, or Snyk.



5 Use .dockerignore File
Exclude unnecessary files and reduce image size.



6 Tag Images Properly
Use meaningful and versioned tags.
(e.g., myapp:v1.0.0)



7 Use Multi-Stage Builds
Build smaller and cleaner images.



8 Manage Secrets Securely
Don't store secrets in images or Dockerfiles.
Use Docker secrets or external tools.



9 Set Resource Limits
Set CPU and memory limits for containers.
Prevents resource exhaustion.



10 Keep Images and Containers Updated
Regularly update base images and rebuild your images.



Dockerfile Example (Good Practice)

```
# Use minimal official image
FROM python:3.11-alpine                          → Small and secure base image
# Set working directory
WORKDIR /app                                     → Set working directory
# Copy only required files
COPY requirements.txt .                          → Install dependencies first
RUN pip install --no-cache-dir -r requirements.txt   (Better layer caching)
# Copy application code
COPY . .                                         → Copy app source
# Create non-root user
RUN adduser -D appuser                           → Run as non-root user
USER appuser
# Run the application
CMD ["python", "app.py"]                         → Default command to run app
```

Real-World Scenario:

A production app image was 1.5GB and had many vulnerabilities. After applying best practices:

- Image size reduced to 180MB
- Zero critical vulnerabilities
- Better performance and faster deployments
- More secure and reliable application



Summary Checklist

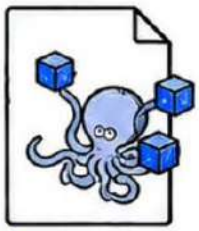
- ☑ Use official, minimal base images
- ☑ Run as non-root user
- ☑ Keep images small
- ☑ Scan images regularly
- ☑ Use .dockerignore
- ☑ Tag images properly
- ☑ Use multi-stage builds
- ☑ Manage secrets securely
- ☑ Set resource limits
- ☑ Keep images updated





Slide 11 - Question 10

Q10. What is Docker Compose?

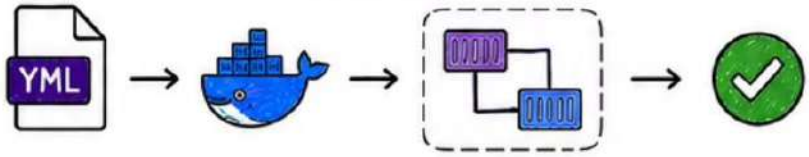


docker-compose.yml

Answer:

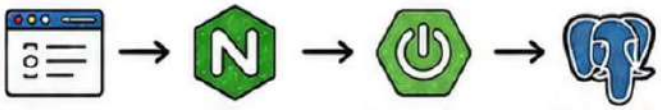
Docker Compose is a tool for defining and running multi-container Docker applications.
It uses a YAML file (docker-compose.yml) to configure the services, networks, and volumes.
With a single command, you can create, start, stop, and manage all the services together.

How it Works



1. Define services in docker-compose.yml
2. Docker Compose reads the file
3. Creates & starts all the services together
4. All services run as one application

Example: Simple Web Application



User → NGINX (Frontend) → Spring Boot (Backend) → PostgreSQL (Database)

Common Commands

1.	`docker compose up`	Start all services
2.	`docker compose up -d`	Start in detached mode
3.	`docker compose down`	Stop and remove services
4.	`docker compose ps`	List running services
5.	`docker compose logs`	View logs
6.	`docker compose build`	Build/rebuild services
7.	`docker compose restart`	Restart services

Sample docker-compose.yml

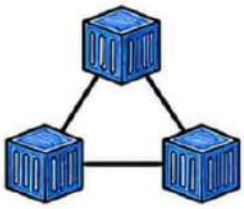
```yaml
version: '3.8'                          # → Compose file version
services:
  web:                                  # → NGINX service (Frontend)
    image: nginx:alpine
    ports:
      - "80:80"
    depends_on:
      - app
  app:                                  # → Spring Boot service (Backend)
    image: my-spring-app:latest
    ports:
      - "8080:8080"
    depends_on:
      - db
  db:                                   # → PostgreSQL service (Database)
    image: postgres:15
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - db_data:/var/lib/postgresql/data   # → Named volume for database
volumes:
  db_data:
```

Real-World Scenario:

Benefits



Slide 12 - Question 11



Q11. What are Docker Networks?

Answer:

Docker Networks allow containers to communicate with each other and with the outside world securely and efficiently.

Docker creates a private network for your containers by default.

Types of Docker Networks

1 bridge (Default)



Used for standalone containers on the same Docker host. Containers can communicate with each other.

2 host



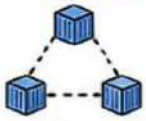
Removes network isolation between container and Docker host. Uses host's network directly.

3 none



Disables all networking for the container. Used for very isolated scenarios.

4 overlay (Swarm)

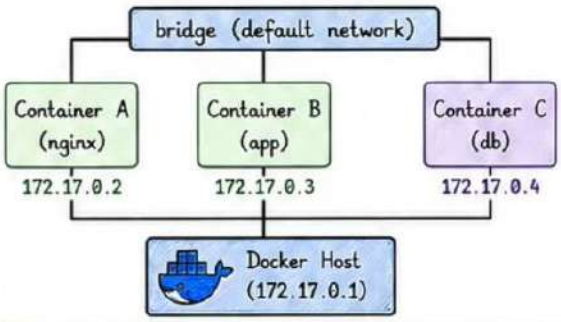


Used in Docker Swarm mode to connect containers across multiple Docker hosts.

Key Docker Network Commands

```
docker network ls                    # List all networks
docker network create <name>         # Create a network
docker network inspect <name>        # Inspect details
docker network rm <name>             # Remove a network
docker network connect <net> <container> # Connect container
docker network disconnect <net> <container> # Disconnect container
```

Default Bridge Network Example



Containers on the same bridge network can talk to each other using their container name or IP address.

Create and Use a Custom Network

1. Create Network

```
docker network create my-net
```

2. Run Containers on the Network

```
docker run -d --name web --network my-net nginx
docker run -d --name app --network my-net myapp
```

3. Containers can communicate using names

```
# From 'web' container, ping 'app'
ping app
```

docker network inspect my-net (Sample Output)

```
{
  "Name": "my-net",
  "Driver": "bridge",
  "Scope": "local",
  "Containers": {
     "a1b2c3": {"Name": "web", "IPv4Address": "172.18.0.2/16"},
     "d4e5f6": {"Name": "app", "IPv4Address": "172.18.0.3/16"}
  },
  "Options": {"com.docker.network.bridge.name": "br-123456"}
}
```

Each network has its own subnet for containers.

Real-World Scenario

In production, we create custom networks to:

Best Practices

- ☑ Use user-defined bridge networks instead of default bridge.